

Lesson 8 Structured Note: C++ Class Implementation of a Linked-List Stack

0. Current Position in the Course

You are now in **Lesson 8** of the stack curriculum.

This lesson belongs to:

Phase 2: Linked-List Stack

Before Lesson 8, you already learned:

Lesson	Main idea
Lesson 1	A stack follows LIFO: Last In, First Out .
Lesson 2	An array stack uses <code>size</code> , <code>top</code> , and an array pointer.
Lesson 3	Array stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 4	A complete C array-stack program combines all array-stack functions.
Lesson 5	A linked-list stack uses nodes and a <code>top</code> pointer.
Lesson 6	Linked-list stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 7	A complete C linked-list stack uses <code>struct Node</code> , a global <code>top</code> pointer, <code>push</code> , <code>pop</code> , <code>display</code> , and <code>main</code> .

Lesson 8 continues the linked-list stack, but now we write it in **C++ using classes**.

The stack rule does not change.

The rule is still:

Last In, First Out

The main change is the coding style.

In Lesson 7, `top` was global.

In Lesson 8, `top` becomes private inside a `Stack` class.

1. Lesson 8 Goal

By the end of this lesson, you should understand:

- How to convert a C linked-list stack into a C++ class-based stack.
 - What a `Node` class is.
 - What a `Stack` class is.
 - Why `top` should be private.
 - What encapsulation means.
 - What a constructor does.
 - How to use the scope resolution operator `::`.
 - How `new` replaces `malloc()` in C++.
 - How `delete` replaces `free()` in C++.
 - How to implement `push()` as a class member function.
 - How to implement `pop()` as a class member function.
 - How to implement `display()` as a class member function.
 - How to add helper functions like `stackTop()` and `isEmpty()`.
 - How to write a destructor to clean up remaining nodes.
 - How to dry-run the complete C++ linked-list stack.
 - The time complexity of each operation.
-

2. Review: What Is a Stack?

A stack is a linear data structure.

It follows this rule:

LIFO = Last In, First Out

This means:

The last value inserted into the stack is the first value removed from the stack.

Example:

```
push(10);  
push(20);  
push(30);
```

Logical stack:

```
Top -> 30  
      20  
      10
```

The last inserted value is **30**.

So if we call:

```
pop();
```

The removed value is:

```
30
```

Important rule:

In a stack, insertion and deletion happen only at the **top**.

3. Review: Linked-List Stack

A linked-list stack stores values using connected nodes.

Example:

```
top -> 30 -> 20 -> 10 -> NULL
```

Here:

- **top** points to the first node.
- The first node contains **30**.
- **30** is the top element.
- **20** is below **30**.
- **10** is at the bottom.

- The last node points to `NULL`.

In a linked-list stack:

`top` is a pointer, not an index.

This is different from an array stack.

Concept	Array stack	Linked-list stack
Storage	Array	Nodes
Meaning of <code>top</code>	Index	Pointer
Empty condition	<code>top == -1</code>	<code>top == NULL</code> or <code>top == nullptr</code>
Full condition	<code>top == size - 1</code>	Heap allocation fails
Push location	End/right side of array	Front of linked list
Pop location	End/right side of array	Front of linked list

4. What Lesson 8 Changes

In Lesson 7, the C version used a global pointer:

```
struct Node *top = NULL;
```

That means `top` was outside all functions.

Because it was global, every function could access it directly.

That is simple, but it is less protected.

In Lesson 8, we move `top` inside a class:

```
class Stack {
private:
    Node* top;
};
```

Now `top` is hidden from outside code.

This is safer.

The main upgrade is:

```
C version:  
Global top pointer  
  
C++ version:  
Private top pointer inside Stack class
```

This idea is called **encapsulation**.

5. What Is Encapsulation?

Encapsulation means:

```
Keep data and the functions that work on that data together.
```

In Lesson 8, the `Stack` class stores:

- the `top` pointer
- `push()`
- `pop()`
- `display()`
- other helper functions

So instead of having loose functions and a global pointer, we keep everything inside one class.

This protects the stack.

Outside code should not directly change `top`.

For example, this should not be allowed:

```
stk.top = nullptr;
```

Why?

Because changing `top` directly from outside could destroy the stack.

Instead, outside code should use stack operations:

```
stk.push(10);  
stk.pop();  
stk.display();
```

This is cleaner and safer.

6. C Version vs C++ Version

C linked-list stack	C++ class linked-list stack
Uses <code>struct Node</code>	Uses <code>class Node</code>
Uses global <code>top</code>	Uses private <code>top</code> inside <code>Stack</code>
Uses <code>malloc()</code>	Uses <code>new</code>
Uses <code>free()</code>	Uses <code>delete</code>
Uses normal functions	Uses member functions
Less protected	More protected
Procedural style	Object-oriented style

Important:

The implementation style changes, but the stack rule does not change.

Both versions still follow:

Last In, First Out

7. The Node Class

A node is one element of the linked list.

Each node stores two things:

1. The value.
2. The address of the next node.

In C++, we can write the node like this:

```
class Node {
public:
    int data;
    Node* next;
};
```

Meaning of Each Field

Field	Meaning
data	Stores the actual stack value.
next	Points to the next node.

Example:

```
[ data = 30 | next ] -> [ data = 20 | next ] -> [ data = 10 | NULL ]
```

The first node stores 30 .

The next node stores 20 .

The final node stores 10 and points to NULL or nullptr .

8. Why Node Members Are Public

In this beginner version, the Node class is written with public members:

```
class Node {
public:
    int data;
    Node* next;
};
```

This allows the Stack class to easily write:

```
t->data = x;
t->next = top;
```

In more advanced C++, we could make Node private or nested inside Stack .

But for this lesson, keeping `Node` simple makes the stack code easier to understand.

9. The Stack Class

The `Stack` class manages the whole stack.

Basic structure:

```
class Stack {  
private:  
    Node* top;  
  
public:  
    Stack();  
    void push(int x);  
    int pop();  
    void display();  
};
```

The class has two sections:

```
private:
```

and

```
public:
```

10. Meaning of `private`

The `private` section contains data that outside code should not directly touch.

In this lesson:

```
private:  
    Node* top;
```

This means:

Only functions inside the `Stack` class can directly use `top`.

For example, these functions can use `top`:

```
Stack::push()  
Stack::pop()  
Stack::display()
```

But code in `main()` cannot directly use `top`.

This is not allowed:

```
stk.top = nullptr;
```

That is good because it prevents accidental damage.

11. Meaning of `public`

The `public` section contains functions that outside code is allowed to use.

Example:

```
public:  
    Stack();  
    void push(int x);  
    int pop();  
    void display();
```

These are the official operations of the stack.

In `main()`, we can write:

```
stk.push(10);  
stk.push(20);  
stk.display();  
stk.pop();
```

This means the user of the class can use the stack without directly touching its internal pointer.

12. Constructor

A constructor is a special function that runs automatically when an object is created.

For the stack, the constructor sets the stack to empty.

Code:

```
Stack::Stack() {  
    top = nullptr;  
}
```

When we write:

```
Stack stk;
```

C++ automatically calls the constructor.

So this happens automatically:

```
top = nullptr;
```

That means the stack starts empty.

Logical view:

```
top -> NULL
```

Why Do We Need the Constructor?

We need the constructor because `top` must have a safe initial value.

If we do not initialize `top`, it may contain garbage memory.

That would make the stack unsafe.

Correct empty state:

```
top = nullptr;
```

Old C-style code may use:

```
top = NULL;
```

Modern C++ usually uses:

```
top = nullptr;
```

Both mean the pointer points to nothing.

13. Scope Resolution Operator ::

In C++, class member functions can be written outside the class body.

When we do that, we use the scope resolution operator:

```
::
```

Example:

```
void Stack::push(int x) {  
    // function body  
}
```

The meaning is:

```
push belongs to the Stack class.
```

Without `Stack::`, C++ would think `push()` is a normal function.

Compare:

```
void push(int x) {  
    // normal function  
}
```

versus:

```
void Stack::push(int x) {  
    // Stack class member function  
}
```

The second one belongs to the `Stack` class.

14. `new` in C++

In C, we used `malloc()` to create a node:

```
t = (struct Node*)malloc(sizeof(struct Node));
```

In C++, we use `new`:

```
Node* t = new Node;
```

This creates one new node on the heap.

The heap is the memory area used for dynamic allocation.

Each successful `push()` creates one new node.

15. `delete` in C++

In C, we used `free()` to release memory:

```
free(p);
```

In C++, we use `delete`:

```
delete p;
```

This releases the memory used by the node.

Each successful `pop()` should delete the removed node.

If we do not delete removed nodes, the program wastes memory.

That problem is called a **memory leak**.

16. malloc/free vs new/delete

C memory function	C++ memory operator	Purpose
<code>malloc()</code>	<code>new</code>	Allocates memory
<code>free()</code>	<code>delete</code>	Releases memory

For this C++ lesson, use:

```
new
```

and

```
delete
```

Do not mix styles in beginner code.

If you allocate with `new`, release with `delete`.

If you allocate with `malloc()`, release with `free()`.

17. push(x) Operation

Meaning

`push(x)` inserts a new value at the top of the stack.

Example before push:

```
top -> 20 -> 10 -> NULL
```

Call:

```
push(30);
```

After push:

```
top -> 30 -> 20 -> 10 -> NULL
```

The new value `30` becomes the top.

18. push(x) Code

```
void Stack::push(int x) {
    Node* t = new Node;

    if (t == nullptr) {
        cout << "Stack Overflow" << endl;
    } else {
        t->data = x;
        t->next = top;
        top = t;
    }
}
```

Note:

In normal C++, `new Node` usually throws an exception if memory allocation fails.

For beginner stack logic, many lessons still show a null check because it matches the linked-list overflow idea.

A safer beginner-friendly version uses:

```
Node* t = new(nothrow) Node;
```

Then `t == nullptr` can be checked directly.

19. push(x) Explanation

Step 1: Create a new node

```
Node* t = new Node;
```

`t` points to the new node.

Step 2: Check whether allocation failed

```
if (t == nullptr)
```

If memory is not available, the stack cannot grow.

That is called:

Stack Overflow

Step 3: Store the value

```
t->data = x;
```

If we call:

```
push(30);
```

then:

```
t->data = 30;
```

Step 4: Link the new node to the old top

```
t->next = top;
```

This makes the new node point to the previous top node.

Step 5: Move top to the new node

```
top = t;
```

Now the new node becomes the top of the stack.

20. The Key push() Pattern

The most important push pattern is:

```
t->next = top;  
top = t;
```

Meaning:

1. The new node points to the old top.
2. Then `top` moves to the new node.

The order matters.

Correct order:

```
t->next = top;  
top = t;
```

Wrong order:

```
top = t;  
t->next = top;
```

The wrong order can damage the stack.

It may make the new node point to itself or lose the old list.

21. push() Dry Run

Start:

```
top -> NULL
```

Call:

```
stk.push(10);
```

Create node `10`.

Since old `top` is `NULL`, node `10` points to `NULL`.

Move `top` to node `10`.

Result:

```
top -> 10 -> NULL
```

Call:

```
stk.push(20);
```

Create node 20.

Node 20 points to old top, which is node 10.

Move top to node 20.

Result:

```
top -> 20 -> 10 -> NULL
```

Call:

```
stk.push(30);
```

Create node 30.

Node 30 points to old top, which is node 20.

Move top to node 30.

Result:

```
top -> 30 -> 20 -> 10 -> NULL
```

Logical stack:

```
Top -> 30  
      20  
      10
```

22. pop() Operation

Meaning

`pop()` removes and returns the top value.

Example before pop:

```
top -> 30 -> 20 -> 10 -> NULL
```

Call:

```
pop();
```

The top value is:

```
30
```

After pop:

```
top -> 20 -> 10 -> NULL
```

Returned value:

```
30
```

23. pop() Code

```
int Stack::pop() {
    int x = -1;
    Node* p;

    if (top == nullptr) {
        cout << "Stack Underflow" << endl;
    } else {
        p = top;
        x = p->data;
        top = top->next;
    }
}
```

```
        delete p;
    }

    return x;
}
```

24. pop() Explanation

Step 1: Prepare a return variable

```
int x = -1;
```

`x` stores the value that will be returned.

If the stack is empty, there is no valid value to return.

So `-1` is used as a simple error value.

Important limitation:

If `-1` is also a valid stack value, then the caller cannot easily know whether `-1` means an error or a real stack value.

For beginner code, this simple method is acceptable.

Step 2: Check underflow

```
if (top == nullptr)
```

If `top == nullptr`, the stack is empty.

There is no node to remove.

This error is called:

Stack Underflow

Step 3: Save the old top node

```
p = top;
```

Now `p` points to the node that will be deleted.

Step 4: Save the data

```
x = p->data;
```

This saves the top value before deleting the node.

Step 5: Move top to the next node

```
top = top->next;
```

Now the second node becomes the new top.

Step 6: Delete the old top node

```
delete p;
```

This releases the memory used by the removed node.

Step 7: Return the saved value

```
return x;
```

25. The Key pop() Pattern

The most important pop pattern is:

```
p = top;  
x = p->data;  
top = top->next;  
delete p;  
return x;
```

Meaning:

1. Save the old top node.
2. Save its value.
3. Move `top` to the next node.
4. Delete the old top node.
5. Return the saved value.

The order matters.

Correct idea:

```
p = top;  
top = top->next;  
delete p;
```

Wrong idea:

```
delete top;  
top = top->next;
```

The wrong idea is unsafe.

After deleting `top`, using `top->next` is invalid.

26. pop() Dry Run

Start:

```
top -> 30 -> 20 -> 10 -> NULL
```

Call:

```
stk.pop();
```

Step 1:

Check whether the stack is empty.

```
top == nullptr
```

This is false because `top` points to node `30`.

Step 2:

Save old top:

```
p = top;
```

Now:

```
p -> 30
```

Step 3:

Save data:

```
x = p->data;
```

So:

```
x = 30
```

Step 4:

Move top:

```
top = top->next;
```

Now:

```
top -> 20 -> 10 -> NULL
```

Step 5:

Delete old node:

```
delete p;
```

The node containing 30 is removed from heap memory.

Step 6:

Return:

30

Final stack:

top -> 20 -> 10 -> NULL

27. display() Operation

Meaning

display() prints all stack elements from top to bottom.

Example stack:

top -> 30 -> 20 -> 10 -> NULL

Output:

30 20 10

It prints 30 first because 30 is the top value.

28. display() Code

```
void Stack::display() {  
    Node* p = top;  
  
    while (p != nullptr) {  
        cout << p->data << " ";  
    }
```

```
        p = p->next;
    }

    cout << endl;
}
```

29. display() Explanation

Step 1: Start a temporary pointer at top

```
Node* p = top;
```

We use `p` because we should not move the real `top` pointer.

Important:

Never move `top` just to display the stack.

If you move `top`, you damage the stack.

Step 2: Traverse until the end

```
while (p != nullptr)
```

The loop continues while `p` points to a valid node.

Step 3: Print current node data

```
cout << p->data << " ";
```

Step 4: Move to the next node

```
p = p->next;
```

The pointer moves through the list:

```
30 -> 20 -> 10 -> NULL
```

When `p` becomes `nullptr`, the loop stops.

30. display() Does Not Change the Stack

Before display:

```
top -> 30 -> 20 -> 10 -> NULL
```

After display:

```
top -> 30 -> 20 -> 10 -> NULL
```

The stack remains the same.

`display()` only reads the stack.

31. stackTop() Helper Function

Meaning

`stackTop()` returns the current top value without deleting it.

It is like `peek(1)`.

Code:

```
int Stack::stackTop() {  
    if (top == nullptr) {  
        return -1;  
    }  
  
    return top->data;  
}
```

Example:

```
top -> 30 -> 20 -> 10 -> NULL
```

Call:

```
stk.stackTop();
```

Return value:

```
30
```

The stack does not change.

32. Difference Between pop() and stackTop()

Operation	Returns top value?	Removes top value?	Changes stack?
<code>pop()</code>	Yes	Yes	Yes
<code>stackTop()</code>	Yes	No	No

Example stack:

```
top -> 30 -> 20 -> 10 -> NULL
```

If we call:

```
stk.stackTop();
```

It returns `30`, but the stack stays:

```
top -> 30 -> 20 -> 10 -> NULL
```

If we call:

```
stk.pop();
```

It returns `30`, and the stack becomes:

```
top -> 20 -> 10 -> NULL
```

33. isEmpty() Helper Function

Meaning

`isEmpty()` checks whether the stack has no nodes.

Code:

```
bool Stack::isEmpty() {  
    return top == nullptr;  
}
```

If:

```
top -> NULL
```

then:

```
isEmpty()
```

returns:

```
true
```

If:

```
top -> 30 -> 20 -> NULL
```

then:

```
isEmpty()
```

returns:

```
false
```

34. Destructor

A destructor is a special function that runs automatically when an object is destroyed.

Constructor creates the initial empty state.

Destructor cleans up memory before the object disappears.

Code:

```
Stack::~~Stack() {
    Node* p;

    while (top != nullptr) {
        p = top;
        top = top->next;
        delete p;
    }
}
```

Why Do We Need a Destructor?

Suppose the program ends while the stack still has nodes:

```
top -> 30 -> 20 -> 10 -> NULL
```

Those nodes were created using `new`.

They should be released using `delete`.

The destructor deletes all remaining nodes.

This prevents memory leaks.

35. Constructor vs Destructor

Function	Runs when?	Purpose
Constructor	When object is created	Initializes <code>top</code> to <code>nullptr</code>
Destructor	When object is destroyed	Deletes remaining nodes

Constructor name:

```
Stack()
```

Destructor name:

```
~Stack()
```

The destructor has a ☐ before the class name.

36. Complete C++ Program

```
#include <iostream>
#include <new>
using namespace std;

class Node {
public:
    int data;
    Node* next;
};

class Stack {
private:
    Node* top;

public:
    Stack();
    ~Stack();

    void push(int x);
    int pop();
    void display();
    int stackTop();
    bool isEmpty();
};

Stack::Stack() {
    top = nullptr;
}

Stack::~Stack() {
    Node* p;

    while (top != nullptr) {
```

```

        p = top;
        top = top->next;
        delete p;
    }
}

void Stack::push(int x) {
    Node* t = new(nothrow) Node;

    if (t == nullptr) {
        cout << "Stack Overflow" << endl;
    } else {
        t->data = x;
        t->next = top;
        top = t;
    }
}

int Stack::pop() {
    int x = -1;
    Node* p;

    if (top == nullptr) {
        cout << "Stack Underflow" << endl;
    } else {
        p = top;
        x = p->data;
        top = top->next;
        delete p;
    }

    return x;
}

void Stack::display() {
    Node* p = top;

    while (p != nullptr) {
        cout << p->data << " ";
        p = p->next;
    }

    cout << endl;
}

int Stack::stackTop() {
    if (top == nullptr) {
        return -1;
    }
}

```

```

    }

    return top->data;
}

bool Stack::isEmpty() {
    return top == nullptr;
}

int main() {
    Stack stk;

    stk.push(10);
    stk.push(20);
    stk.push(30);

    cout << "Stack: ";
    stk.display();

    cout << "Popped: " << stk.pop() << endl;

    cout << "Stack after pop: ";
    stk.display();

    cout << "Top element: " << stk.stackTop() << endl;

    if (stk.isEmpty()) {
        cout << "Stack is empty" << endl;
    } else {
        cout << "Stack is not empty" << endl;
    }

    return 0;
}

```

37. Explanation of the Complete Program

Step 1: Include header files

```

#include <iostream>
#include <new>
using namespace std;

```

`iostream` is used for:

cout

`new` is included for:

nothrow

`nothrow` lets us check whether `new` failed by comparing the result with `nullptr`.

Step 2: Define the Node class

```
class Node {  
public:  
    int data;  
    Node* next;  
};
```

Each node stores:

- one integer value
- one pointer to the next node

Step 3: Define the Stack class

```
class Stack {  
private:  
    Node* top;  
  
public:  
    Stack();  
    ~Stack();  
    void push(int x);  
    int pop();  
    void display();  
    int stackTop();  
    bool isEmpty();  
};
```

`top` is private.

The functions are public.

Step 4: Constructor initializes the stack

```
Stack::Stack() {  
    top = nullptr;  
}
```

This makes the stack empty at the start.

Step 5: Push values

```
stk.push(10);  
stk.push(20);  
stk.push(30);
```

After these calls:

```
top -> 30 -> 20 -> 10 -> NULL
```

Step 6: Display stack

```
stk.display();
```

Output:

```
30 20 10
```

Step 7: Pop one value

```
stk.pop();
```

This removes `30`.

Now the stack is:

```
top -> 20 -> 10 -> NULL
```

Step 8: Check top value

```
stk.stackTop();
```

Now the top value is:

```
20
```

Step 9: Check whether stack is empty

```
stk.isEmpty();
```

The stack has two nodes, so the answer is:

```
false
```

Step 10: Destructor runs automatically

When `main()` ends, `stk` is destroyed.

The destructor deletes any remaining nodes.

38. Expected Output

A possible output is:

```
Stack: 30 20 10
Popped: 30
Stack after pop: 20 10
Top element: 20
Stack is not empty
```

39. Full Dry Run

Program operations:

```
Stack stk;  
stk.push(10);  
stk.push(20);  
stk.push(30);  
stk.display();  
stk.pop();  
stk.display();  
stk.stackTop();  
stk.isEmpty();
```

Initial object creation

```
Stack stk;
```

Constructor runs automatically.

Result:

```
top -> NULL
```

Operation 1: push(10)

Create node 10.

Node 10 points to old top, which is NULL.

Move top to node 10.

Result:

```
top -> 10 -> NULL
```

Operation 2: push(20)

Create node 20.

Node 20 points to old top, which is node 10.

Move top to node 20.

Result:

```
top -> 20 -> 10 -> NULL
```

Operation 3: push(30)

Create node 30.

Node 30 points to old top, which is node 20.

Move top to node 30.

Result:

```
top -> 30 -> 20 -> 10 -> NULL
```

Operation 4: display()

Start p at top.

Print 30.

Move p to 20.

Print 20.

Move p to 10.

Print 10.

Move p to NULL.

Stop.

Output:

```
30 20 10
```

The stack does not change.

Operation 5: pop()

Current stack:

```
top -> 30 -> 20 -> 10 -> NULL
```

Save old top in `p`.

Save data `30` in `x`.

Move top to node `20`.

Delete old node `30`.

Return `30`.

Result:

```
top -> 20 -> 10 -> NULL
```

Operation 6: display()

Output:

```
20 10
```

Operation 7: stackTop()

Current top node contains `20`.

Return:

```
20
```

Operation 8: isEmpty()

Check:

```
top == nullptr
```

This is false.

Return:

false

40. Dry Run Summary Table

Step	Operation	Return/output	Stack after operation
0	Stack stk	Constructor runs	top -> NULL
1	push(10)	—	top -> 10 -> NULL
2	push(20)	—	top -> 20 -> 10 -> NULL
3	push(30)	—	top -> 30 -> 20 -> 10 -> NULL
4	display()	30 20 10	top -> 30 -> 20 -> 10 -> NULL
5	pop()	returns 30	top -> 20 -> 10 -> NULL
6	display()	20 10	top -> 20 -> 10 -> NULL
7	stackTop()	returns 20	top -> 20 -> 10 -> NULL
8	isEmpty()	returns false	top -> 20 -> 10 -> NULL

41. Overflow and Underflow

Overflow

Overflow means the stack cannot accept another value.

In a linked-list stack, overflow happens when heap memory cannot create a new node.

In C++ beginner code:

```
Node* t = new(nothrow) Node;

if (t == nullptr) {
    cout << "Stack Overflow" << endl;
}
```

Simple meaning:

Stack overflow means new node allocation failed.

Underflow

Underflow means trying to remove a value from an empty stack.

In a linked-list stack, empty means:

```
top == nullptr
```

Code:

```
if (top == nullptr) {  
    cout << "Stack Underflow" << endl;  
}
```

Simple meaning:

Stack underflow means there is no node to remove.

42. Memory Behavior

A linked-list stack uses heap memory.

Every successful `push()` creates one node:

```
Node* t = new Node;
```

Every successful `pop()` deletes one node:

```
delete p;
```

If the stack still has nodes when the object is destroyed, the destructor deletes them:

```
while (top != nullptr) {  
    p = top;  
    top = top->next;
```

```
    delete p;  
}
```

Important rule:

Every node created with new should eventually be released with delete.

43. Time Complexity

Operation	Time complexity	Reason
Constructor	$O(1)$	Sets <code>top = nullptr</code> .
<code>push()</code>	$O(1)$	Adds one node at the front.
<code>pop()</code>	$O(1)$	Removes one node from the front.
<code>stackTop()</code>	$O(1)$	Reads <code>top->data</code> directly.
<code>isEmpty()</code>	$O(1)$	Checks whether <code>top == nullptr</code> .
<code>display()</code>	$O(n)$	Visits every node.
Destructor	$O(n)$	Deletes every remaining node.

44. Why push() Is $O(1)$

`push()` does a fixed number of steps:

1. Create one node.
2. Store the value.
3. Link the new node to the old top.
4. Move `top` to the new node.

It does not scan the whole list.

So:

`push() =  $O(1)$`

45. Why pop() Is O(1)

`pop()` directly removes the node pointed to by `top`.

It does not search for the top node.

It already has direct access to the top node.

So:

```
pop() = O(1)
```

46. Why display() Is O(n)

`display()` must print every node.

Example:

```
top -> 50 -> 40 -> 30 -> 20 -> 10 -> NULL
```

To print all values, it must visit:

```
50, 40, 30, 20, 10
```

If there are `n` nodes, it visits `n` nodes.

So:

```
display() = O(n)
```

47. Why Destructor Is O(n)

The destructor deletes all remaining nodes.

If the stack has `n` nodes, it must delete `n` nodes.

So:

Destructor = $O(n)$

48. Space Complexity

Each node stores:

- data
- next

If the stack has n values, it has n nodes.

So total stack storage is:

$O(n)$

But each individual operation like `push()` or `pop()` uses only a few temporary variables.

So auxiliary space per operation is:

$O(1)$

49. Important Beginner Mistakes

Mistake 1: Forgetting to initialize top

Wrong:

```
Stack::Stack() {  
    // top not initialized  
}
```

Correct:

```
Stack::Stack() {  
    top = nullptr;  
}
```

If `top` is not initialized, it may contain garbage memory.

Mistake 2: Making top public

Weak design:

```
class Stack {  
public:  
    Node* top;  
};
```

Better design:

```
class Stack {  
private:  
    Node* top;  
};
```

`top` should be private because outside code should not directly change it.

Mistake 3: Reversing the push links

Wrong:

```
top = t;  
t->next = top;
```

Correct:

```
t->next = top;  
top = t;
```

Connect the new node to the old stack first.

Then move `top`.

Mistake 4: Deleting before moving top

Wrong:

```
delete top;
top = top->next;
```

Correct:

```
p = top;
top = top->next;
delete p;
```

After deleting a node, you should not use it.

Mistake 5: Moving top in display()

Wrong:

```
while (top != nullptr) {
    cout << top->data << " ";
    top = top->next;
}
```

This destroys the stack because it moves the real `top` pointer.

Correct:

```
Node* p = top;

while (p != nullptr) {
    cout << p->data << " ";
    p = p->next;
}
```

Use a temporary pointer for traversal.

Mistake 6: Forgetting delete

Wrong:

```
p = top;
top = top->next;
return p->data;
```

This removes the node logically but does not release memory.

Correct:

```
p = top;
x = p->data;
top = top->next;
delete p;
return x;
```

Mistake 7: Confusing `.` and `->`

Use `.` when you have an object.

Example:

```
stk.push(10);
```

Use `->` when you have a pointer.

Example:

```
t->data = x;
t->next = top;
```

Simple rule:

Situation	Operator
Object	<code>.</code>
Pointer	<code>-></code>

50. Key Code Patterns to Memorize

Constructor

```
Stack::Stack() {  
    top = nullptr;  
}
```

Push

```
t->data = x;  
t->next = top;  
top = t;
```

Pop

```
p = top;  
x = p->data;  
top = top->next;  
delete p;  
return x;
```

Display

```
Node* p = top;  
  
while (p != nullptr) {  
    cout << p->data << " ";  
    p = p->next;  
}
```

Stack top

```
return top->data;
```

Empty check

```
return top == nullptr;
```

Destructor

```
while (top != nullptr) {  
    p = top;  
    top = top->next;  
    delete p;  
}
```

51. Lesson 8 Summary

Lesson 8 teaches how to implement a linked-list stack using a C++ class.

The stack behavior stays the same:

Last In, First Out

The main improvement is better structure and safety.

Instead of this C-style approach:

```
struct Node* top = NULL;
```

we use this C++ class-based approach:

```
class Stack {  
private:  
    Node* top;  
};
```

This hides `top` from outside code.

That is encapsulation.

The important C++ ideas are:

- `class Node` represents one node.
- `class Stack` manages the whole stack.
- `private` protects `top`.
- `public` exposes safe stack operations.
- The constructor initializes `top`.

- The destructor cleans up memory.
 - `new` creates nodes.
 - `delete` removes nodes.
 - `Stack::push` means `push` belongs to the `Stack` class.
-

52. Beginner Checkpoints

Before moving to Lesson 9, you should be able to answer these:

1. What is the difference between a global `top` pointer and a private `top` pointer?
 2. Why should `top` be private?
 3. What does the constructor do?
 4. What does the destructor do?
 5. What does `Stack::push` mean?
 6. Why does `push()` use `new`?
 7. Why does `pop()` use `delete`?
 8. Why does `display()` use a temporary pointer?
 9. Why is `push()` $O(1)$?
 10. Why is `display()` $O(n)$?
-

53. Practice Tasks

Task 1

Write the `Node` class from memory.

Task 2

Write the `Stack` class declaration from memory.

Include:

- private `top`
- constructor
- destructor
- `push()`
- `pop()`
- `display()`
- `stackTop()`
- `isEmpty()`

Task 3

Dry-run this sequence:


```
stk.push(5);
stk.push(12);
stk.push(7);
stk.pop();
stk.push(20);
stk.display();
```

Expected final stack:

```
top -> 20 -> 12 -> 5 -> NULL
```

Expected display:

```
20 12 5
```

Task 4

Explain in your own words why this order is correct:

```
t->next = top;
top = t;
```

Task 5

Explain in your own words why this order is safe:

```
p = top;
top = top->next;
delete p;
```

54. Final Takeaway

Lesson 8 is not about changing the stack rule.

It is about improving the implementation.

The C version works, but it uses a global `top` pointer.

The C++ version is cleaner because it hides `top` inside a class.

That gives us better organization, better safety, and a stronger foundation for future stack applications.

The core idea is:

Use the Stack class to control the stack.
Do not let outside code directly control top.